Senior Backend Engineer — Evaluation Assignment

Smart Maritime Document Extractor (SMDE)

Role: Senior Backend Engineer
Time Estimate: 4–6 hours
Submission Deadline: 5 days from receipt
Submission Format: GitHub repository + Architecture Decision Record (ADR) document + 10–15 min Loom walkthrough

Submission: Email your completed assignment (GitHub repo + demo video) to hiring@**skycladventures.com**

What This Assignment Is Testing

This is not a pure coding challenge. We are hiring someone who will both write backend code and lead the team building this product. The submission has three parts that are weighted equally:

1. **The working service** — a backend API that processes maritime documents through an LLM pipeline
2. **A written Architecture Decision Record** — explaining the choices you made and the tradeoffs you rejected
3. **A code review response** — you will receive a junior engineer's PR to review; your written feedback is evaluated as leadership output

We are looking for evidence of three things across all three parts:

- Technical depth — you understand the hard parts and handle them deliberately
- Architectural judgment — you make defensible decisions and know what you are deferring

- Leadership communication — your writing is clear enough to grow a junior engineer and align a product manager

---

Context

We are building a system that allows maritime Manning Agents to upload seafarer certification documents — certificates, medical exams, passports, drug tests — and extract structured data from them automatically using a vision-capable LLM.

Instead of building a rigid parser with hardcoded fields per document type, we use an LLM to:

1. Detect what type of document was uploaded
2. Determine whether it belongs to a Deck or Engine officer
3. Extract the relevant fields dynamically based on what the document actually contains

The product is in active development. A junior engineer has already built a rough prototype (the PR you will review is theirs). You are joining to lead the backend, architect the production system, and grow the team. This assignment simulates week one.

Part 1 — The Service

What You Are Building

A production-oriented backend service with the following capabilities:

Endpoint	Method	Purpose
`POST /api/extract`	Sync or async	Accept a document, extract structured data via LLM
`GET /api/jobs/:jobId`	GET	Poll the status and result of an async extraction job
`GET /api/sessions/:sessionId`	GET	Return all extraction records for a session
`POST /api/sessions/:sessionId/validate`	POST	Cross-document compliance validation
`GET /api/sessions/:sessionId/report`	GET	Return a structured compliance report for the session
`GET /api/health`	GET	Health check endpoint with dependency status

Stack

- **Runtime:** Node.js (TypeScript strongly preferred) or Python (FastAPI)
- **Database:** PostgreSQL or SQLite. If PostgreSQL, provide a docker-compose.yml for the DB only.
- **Queue:** Any — BullMQ, pg-boss, a simple in-process queue, or even a naive polling table. The mechanism matters less than the reasoning behind your choice.
- **LLM Provider:** Your choice — see table below

LLM Provider

Use whichever provider you have access to. What matters is that the model supports vision (image input) and handles base64-encoded documents.

Provider	Free Tier	Recommended Model
Anthropic Claude	Free credits on signup at console.anthropic.com	`claude-haiku-4-5-20251001`
Google Gemini	Generous free quota, no card at aistudio.google.com	`gemini-2.0-flash`
Groq	Free tier at console.groq.com	`llama-3.2-11b-vision-preview`
Mistral	Free credits at console.mistral.ai	`pixtral-12b-2409`
OpenAI	Paid only	`gpt-4o-mini`
Ollama (local)	Completely free, runs locally	`llava` or `llama3.2-vision`

> If you have nothing set up, Google AI Studio (Gemini) and Groq both offer free tiers with no credit card required.

Hard requirement: LLM provider, model, and API key must be configurable via environment variables (`LLM_PROVIDER`, `LLM_MODEL`, `LLM_API_KEY`). The service must not require code changes to swap providers.

Endpoint Specifications

POST /api/extract

This endpoint must support **two modes**, selected by a query parameter:

- `?mode=sync` — process immediately, block until done, return the full extraction result in the response (default behavior for small files)
- `?mode=async` — accept the upload, enqueue the job, return `202 Accepted` immediately with a `jobId`

Request

- Content-Type: `multipart/form-data`
- Field: `document` (file)

- Field: `sessionId` (string, optional — if omitted, create a new session)
- Accepted types: `image/jpeg`, `image/png`, `application/pdf`
- Max size: 10MB

Sync response — 200

```
{ "id": "uuid", "sessionId": "uuid", "fileName": "PEME_Samoya.pdf",
"documentType": "PEME", "documentName": "Pre-Employment Medical Exami
nation", "applicableRole": "ENGINE", "category": "MEDICAL", "confiden
ce": "HIGH", "holderName": "Samuel P. Samoya", "dateOfBirth": "12/03/
1988", "sirbNumber": "C0869326", "passportNumber": null, "fields":
[...], "validity": { "dateOfIssue": "06/01/2025", "dateOfExpiry": "0
6/01/2027", "isExpired": false, "daysUntilExpiry": 660, "revalidation
Required": false }, "compliance": {...}, "medicalData": { "fitnessRes
ult": "FIT", "drugTestResult": "NEGATIVE", "restrictions": null, "spe
cialNotes": "Schistosomiasis — cleared by hematologist. Cardiac dysrh
ythmia — Class B, cleared by cardiologist.", "expiryDate": "06/01/202
7" }, "flags": [...], "isExpired": false, "processingTimeMs": 4230,
"summary": "...", "createdAt": "2026-03-17T08:42:11Z" }
```

Async response — 202

```
{ "jobId": "uuid", "sessionId": "uuid", "status": "QUEUED", "pollUr
l": "/api/jobs/uuid", "estimatedWaitMs": 6000 }
```

Deduplication: If the same file (matched by SHA-256 hash) is uploaded to the same session, return the existing extraction result immediately with a `200` and a header `X-Deduplicated: true`. Do not call the LLM again.

Error responses

Status	Code	Condition
400	UNSUPPORTED_FORMAT	File type not accepted
400	INSUFFICIENT_DOCUMENTS	Validate called with fewer than 2 documents
413	FILE_TOO_LARGE	File exceeds 10MB
404	SESSION_NOT_FOUND	Session ID does not exist
404	JOB_NOT_FOUND	Job ID does not exist
422	LLM_JSON_PARSE_FAIL	LLM returned unparseable response after retry
429	RATE_LIMITED	Too many requests — see rate limiting section
500	INTERNAL_ERROR	Unexpected server error

All errors follow this shape:

```
{ "error": "LLM_JSON_PARSE_FAIL", "message": "Document extraction failed after retry. The raw response has been stored for review.", "extractionId": "uuid-of-failed-record", "retryAfterMs": null }
```

GET /api/jobs/:jobId

Polls the status of an async extraction job.

States: `QUEUED` → `PROCESSING` → `COMPLETE` | `FAILED`

Response while processing — 200

```
{ "jobId": "uuid", "status": "PROCESSING", "queuePosition": 2, "start
edAt": "2026-03-17T08:42:00Z", "estimatedCompleteMs": 3200 }
```

Response when complete — 200

```
{ "jobId": "uuid", "status": "COMPLETE", "extractionId": "uuid", "res
ult": { ... }, "completedAt": "2026-03-17T08:42:11Z" }
```

Response when failed — 200

```
{ "jobId": "uuid", "status": "FAILED", "error": "LLM_JSON_PARSE_FAI
L", "message": "...", "failedAt": "2026-03-17T08:42:11Z", "retryabl
e": true }
```

GET /api/sessions/:sessionId

Returns a summary of all documents in the session.

```
{ "sessionId": "uuid", "documentCount": 5, "detectedRole": "DECK", "o
verallHealth": "WARN", "documents": [ { "id": "uuid", "fileName": "CO
C_Salonoy.jpg", "documentType": "COC", "applicableRole": "DECK", "hol
derName": "Francisco J. Salonoy", "confidence": "HIGH", "isExpired":
false, "flagCount": 0, "criticalFlagCount": 0, "createdAt": "2026-03-
17T08:40:00Z" } ], "pendingJobs": [] }
```

`overallHealth` is derived client-side from the session data — `OK` if no expired certs and no CRITICAL flags, `WARN` if any MEDIUM or HIGH flags or certs expiring within 90 days, `CRITICAL` if any CRITICAL flags or expired required certs.

POST /api/sessions/:sessionId/validate

Sends all extraction records from the session to the LLM for cross-document compliance assessment.

You must write the LLM prompt for this endpoint yourself. We provide the extraction prompt for `/api/extract` (see below) but the validation prompt is your design. This is intentional — we want to see how you instruct an LLM to reason about compliance across multiple heterogeneous documents.

The response must include:

```
{ "sessionId": "uuid", "holderProfile": { ... }, "consistencyChecks":
[ ... ], "missingDocuments": [ ... ], "expiringDocuments": [ ... ],
"medicalFlags": [ ... ], "overallStatus": "APPROVED | CONDITIONAL | R
EJECTED", "overallScore": 74, "summary": "...", "recommendations": [
"...", "..." ], "validatedAt": "2026-03-17T08:45:00Z" }
```

GET /api/sessions/:sessionId/report

Returns a structured, human-readable compliance report for the session. This is not another LLM call — it is derived entirely from data already in the database (extraction records + the most recent validation result).

The shape is yours to design. Think about what a Manning Agent actually needs to see to make a hire/no-hire decision. We will evaluate the structure and completeness of your report schema as a product thinking signal.

GET /api/health

```
{ "status": "OK", "version": "1.0.0", "uptime": 3612, "dependencies":
{ "database": "OK", "llmProvider": "OK", "queue": "OK" }, "timestam
p": "2026-03-17T08:45:00Z" }
```

Rate Limiting

Implement rate limiting on `POST /api/extract` only. The limit is **10 requests per minute per IP**. When the limit is exceeded, return `429 RATE_LIMITED` with a `Retry-After` header and `retryAfterMs` in the body.

You may use any mechanism — an in-memory token bucket, a Redis counter, a middleware library. Document your choice in the ADR.

LLM Reliability Requirements

The LLM is the highest-risk component in the pipeline. Your implementation must handle all of the following:

1. **Malformed JSON** — the LLM sometimes wraps the response in a markdown code fence or adds an explanation before the JSON object. Extract valid JSON by locating the outermost `{` and `}` regardless of surrounding text.
2. **Parse failure recovery** — if extraction fails after the boundary approach, send a repair prompt to the LLM with the raw response and ask it to return clean JSON. Store the raw response regardless.
3. **Timeout handling** — set a 30-second timeout on the LLM API call. On timeout, mark the job `FAILED` with `retryable: true`. Do not hang the request.
4. **LOW confidence retry** — if the LLM returns `confidence: "LOW"`, automatically retry once with a more focused prompt that includes the file name and MIME type as hints. Use the higher-confidence result if the retry succeeds.
5. **Never discard** — even on total failure, store a record with `status: FAILED` and the raw LLM response (or error). Nothing uploaded by the user is ever silently lost.

The LLM Extraction Prompt

Use the following prompt for `POST /api/extract`. Do not modify it — consistent prompt usage across candidates lets us compare extraction quality.

```
You are an expert maritime document analyst with deep knowledge of ST
CW, MARINA, IMO, and international seafarer certification standards.
A document has been provided. Perform the following in a single pass:
1. IDENTIFY the document type from the taxonomy below 2. DETERMINE if
this belongs to a DECK officer, ENGINE officer, BOTH, or is role-agno
stic (N/A) 3. EXTRACT all fields that are meaningful for this specifi
c document type 4. FLAG any compliance issues, anomalies, or concerns
Document type taxonomy (use these exact codes): COC | COP_BT | COP_PS
CRB | COP_AFF | COP_MEFA | COP_MECA | COP_SSO | COP_SDSD | ECDIS_GENE
RIC | ECDIS_TYPE | SIRB | PASSPORT | PEME | DRUG_TEST | YELLOW_FEVER
| ERM | MARPOL | SULPHUR_CAP | BALLAST_WATER | HATCH_COVER | BRM_SSBT
| TRAIN_TRAINER | HAZMAT | FLAG_STATE | OTHER Return ONLY a valid JSO
N object. No markdown. No code fences. No preamble. { "detection": {
"documentType": "SHORT_CODE", "documentName": "Full human-readable do
cument name", "category": "IDENTITY | CERTIFICATION | STCW_ENDORSEMEN
T | MEDICAL | TRAINING | FLAG_STATE | OTHER", "applicableRole": "DECK
| ENGINE | BOTH | N/A", "isRequired": true, "confidence": "HIGH | MED
IUM | LOW", "detectionReason": "One sentence explaining how you ident
ified this document" }, "holder": { "fullName": "string or null", "da
teOfBirth": "DD/MM/YYYY or null", "nationality": "string or null", "p
assportNumber": "string or null", "sirbNumber": "string or null", "ra
nk": "string or null", "photo": "PRESENT | ABSENT" }, "fields": [ {
"key": "snake_case_key", "label": "Human-readable label", "value": "e
xtracted value as string", "importance": "CRITICAL | HIGH | MEDIUM |
LOW", "status": "OK | EXPIRED | WARNING | MISSING | N/A" } ], "validi
ty": { "dateOfIssue": "string or null", "dateOfExpiry": "string | 'No
Expiry' | 'Lifetime' | null", "isExpired": false, "daysUntilExpiry":
null, "revalidationRequired": null }, "compliance": { "issuingAuthori
ty": "string", "regulationReference": "e.g. STCW Reg VI/1 or null",
"imoModelCourse": "e.g. IMO 1.22 or null", "recognizedAuthority": tru
e, "limitations": "string or null" }, "medicalData": { "fitnessResul
t": "FIT | UNFIT | N/A", "drugTestResult": "NEGATIVE | POSITIVE | N/
A", "restrictions": "string or null", "specialNotes": "string or nul
l", "expiryDate": "string or null" }, "flags": [ { "severity": "CRITI
CAL | HIGH | MEDIUM | LOW", "message": "Description of issue or conce
rn" } ], "summary": "Two-sentence plain English summary of what this
document confirms about the holder." }
```

Database Schema

Design your own schema. The suggested schema below is a starting point only — we will evaluate your schema design as part of the technical review. Consider: what indexes will you need? How do you model job state? How do you avoid JSONB columns becoming a dumping ground?

```
-- Starting point only — modify freely CREATE TABLE sessions ( id TEX
T PRIMARY KEY, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); CREA
TE TABLE extractions ( id TEXT PRIMARY KEY, session_id TEXT NOT NULL
REFERENCES sessions(id), file_name TEXT NOT NULL, file_hash TEXT NOT
NULL, document_type TEXT, applicable_role TEXT, confidence TEXT, hold
er_name TEXT, date_of_birth TEXT, sirb_number TEXT, passport_number T
EXT, fields_json TEXT, validity_json TEXT, medical_data_json TEXT, fl
ags_json TEXT, is_expired INTEGER DEFAULT 0, summary TEXT, raw_llm_re
sponse TEXT, processing_time_ms INTEGER, status TEXT DEFAULT 'COMPLET
E', created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE jo
bs ( id TEXT PRIMARY KEY, session_id TEXT REFERENCES sessions(id), ex
traction_id TEXT REFERENCES extractions(id), status TEXT DEFAULT 'QUE
UED', error_code TEXT, error_message TEXT, queued_at TIMESTAMP DEFAUL
T CURRENT_TIMESTAMP, started_at TIMESTAMP, completed_at TIMESTAMP );
CREATE TABLE validations ( id TEXT PRIMARY KEY, session_id TEXT NOT N
ULL REFERENCES sessions(id), result_json TEXT NOT NULL, created_at TI
MESTAMP DEFAULT CURRENT_TIMESTAMP );
```

Part 2 — Architecture Decision Record

Write a short ADR document (500–800 words, markdown is fine, include it as `ADR.md` in the repo root) covering the following questions. Answer each one directly — we are not looking for hedged non-answers.

Question 1 — Sync vs Async
You implemented a `?mode=sync` and `?mode=async` option. In production, which mode should be the default and why? At what file size or concurrency threshold would you force async regardless of the mode param?

Question 2 — Queue Choice
What queue mechanism did you use and why? What would you migrate to if this service needed to handle 500 concurrent extractions per minute? What are the failure modes of your current approach?

Question 3 — LLM Provider Abstraction
Did you build a provider interface that makes swapping LLMs trivial, or did you implement against one provider directly? Justify the decision. If you did build the abstraction, describe its interface.

Question 4 — Schema Design
The suggested schema uses JSONB/TEXT columns for dynamic fields. What are the risks of that approach at scale? What would you change if this system needed to support full-text search across extracted field values, or querying "all sessions where any document has an expired COC"?

Question 5 — What You Skipped
What did you deliberately not implement that a production system would require? List at least three things and briefly explain why you deprioritized each.

Part 3 — Code Review

Below is a pull request submitted by a junior engineer on the team. Review it as you would in a real code review — written comments on specific lines or sections, a summary comment at the top explaining your overall assessment, and at least one thing you would specifically call out as a teaching moment.

Your review will be evaluated on: technical accuracy, tone (is it constructive?), whether you identify the real problems vs. surface issues, and whether your feedback would actually help a junior engineer grow.

The PR: "feat: add document extraction endpoint"

```
// src/routes/extract.ts import express from 'express'; import Anthro
pic from '@anthropic-ai/sdk'; import fs from 'fs'; import path from
'path'; const router = express.Router(); const client = new Anthropic
({ apiKey: 'sk-ant-REDACTED' }); router.post('/extract', async (req,
res) => { const file = req.file; if (!file) { res.status(400).json({
error: 'No file uploaded' }); return; } try { // Read the file and co
nvert to base64 const fileData = fs.readFileSync(file.path); const ba
se64Data = fileData.toString('base64'); // Save file to disk permanen
tly for reference const savedPath = path.join('./uploads', file.origi
nalname); fs.copyFileSync(file.path, savedPath); const response = awa
it client.messages.create({ model: 'claude-opus-4-6', max_tokens: 409
6, messages: [ { role: 'user', content: [ { type: 'image', source: {
type: 'base64', media_type: file.mimetype, data: base64Data, }, }, {
type: 'text', text: 'Extract all information from this maritime docum
ent and return as JSON.', }, ], }, ], }); const result = JSON.parse(r
esponse.content[0].text); // Store in memory for now global.extractio
ns = global.extractions || []; global.extractions.push(result); res.j
son(result); } catch (error) { console.log('Error:', error); res.stat
us(500).json({ error: 'Something went wrong' }); } }); export default
router;
```

PR description written by the junior engineer:

> "Added the main extract endpoint. It reads the uploaded file, converts to base64, sends to Claude, parses the JSON back, and returns it. Also saves files to disk so we don't lose them. Tested with one PEME file and it worked. Using Opus because it gave better results in my testing."

Submit your code review as `CODE_REVIEW.md` in the repo root. Write it as if you are posting it in GitHub — address the junior engineer directly.

Evaluation Criteria

Technical (50%)

Area	What We Look For
LLM reliability	JSON extraction robustness, repair strategy, no silent failures, raw response always stored
Async pipeline	Job state machine is correct, polling endpoint handles all states, queue does not drop jobs on restart
API design	Consistent error shapes, sync/async modes work correctly, deduplication works, rate limiting returns correct headers
Schema design	Indexes exist where needed, job table is correct, schema choices are defensible in the ADR
Code quality	TypeScript types are real (no `any` dumps), separation of concerns, no hardcoded credentials, README runs in 3 commands
Testing	At minimum: JSON repair logic has unit tests, happy path has an integration test or Postman collection

Architecture & Judgment (25%)

Area	What We Look For
ADR quality	Direct answers, real tradeoffs named, honest about what was skipped
Report endpoint design	Does the schema reflect genuine product thinking about what a Manning Agent needs?
Validation prompt	Is the cross-document compliance prompt precise, well-structured, and would it produce reliable output?
Schema evolution	Did you think about query patterns beyond just "store and retrieve"?

Leadership (25%)

Area	What We Look For
Code review tone	Constructive and specific — does not berate, does not approve bad code uncritically
Code review accuracy	Identifies the real issues (hardcoded key, Opus cost, global state, no timeout, broad prompt, saved files with PII) not just style nits
Teaching quality	At least one comment explains *why* something is wrong, not just *that* it is wrong
Loom walkthrough	Can you explain architectural decisions clearly to a non-technical audience in 2 minutes and a technical one in 5?

What a Lead-Level Submission Looks Like

A strong submission will:

- Make a clear, justified call on sync vs async default and document it
- Have a job state machine that handles `QUEUED → PROCESSING → COMPLETE/FAILED` correctly with no orphaned jobs
- Write a cross-document validation prompt that would genuinely work in production — specific, structured, and resistant to hallucination
- Design a `/report` endpoint schema that a product manager could read without translation
- Deliver a code review that a junior engineer would actually learn from — specific